```
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import (
    preprocess_input,
    decode_predictions
)

from tensorflow.keras.preprocessing import image
from google.colab import files

# 1. Load pretrained CNN
model = MobileNetV2(weights="imagenet")

print("CNN model loaded successfully!")

# 2. Upload an animal image
uploaded = files.upload()

filename = list(uploaded.keys())[0]

print("Uploaded image:", filename)

# 3. Display original image
original_image = image.load_img(filename)

plt.imshow(original_image)
```

```
plt.title("Uploaded Animal Image")
plt.axis("off")
plt.show()

# 4. Resize image for MobileNetV2
img = image.load_img(
    filename,
    target_size=(224, 224)
)

# 5. Convert image into an array
img_array = image.img_to_array(img)

# 6. Add batch dimension
img_array = np.expand_dims(
    img_array,
    axis=0
)

# 7. Preprocess image
img_array = preprocess_input(img_array)

# 8. Make prediction
predictions = model.predict(img_array)

# 9. Decode top three predictions
results = decode_predictions(
    predictions,
```

```
    top=3
)[0]

# 10. Print predictions
print("\nTop 3 Predictions:")

for number, result in enumerate(results, start=1):

    predicted_name = result[1].replace("_", " ")
    confidence = result[2] * 100

    print(
        f"{number}. {predicted_name}: "
        f"{confidence:.2f}%"
    )

# 11. Get best prediction
best_name = results[0][1].replace("_", " ")
best_confidence = results[0][2] * 100

# 12. Display final result
plt.imshow(original_image)

plt.title(
    f"Prediction: {best_name}\n"
    f"Confidence: {best_confidence:.2f}%"
)
```

plt.axis("off")

plt.show()

OUTPUT;

CNN model loaded successfully!

 **Screenshot 2026-07-25 092024.png**(image/png) - 496499 bytes, last modified: 7/25/2026 - 100% done

Saving Screenshot 2026-07-25 092024.png to Screenshot 2026-07-25 092024 (2).png

Uploaded image: Screenshot 2026-07-25 092024 (2).png


WARNING:tensorflow:5 out of the last 5 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x793961b07e20> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

**1/1** ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **1s** 1s/step

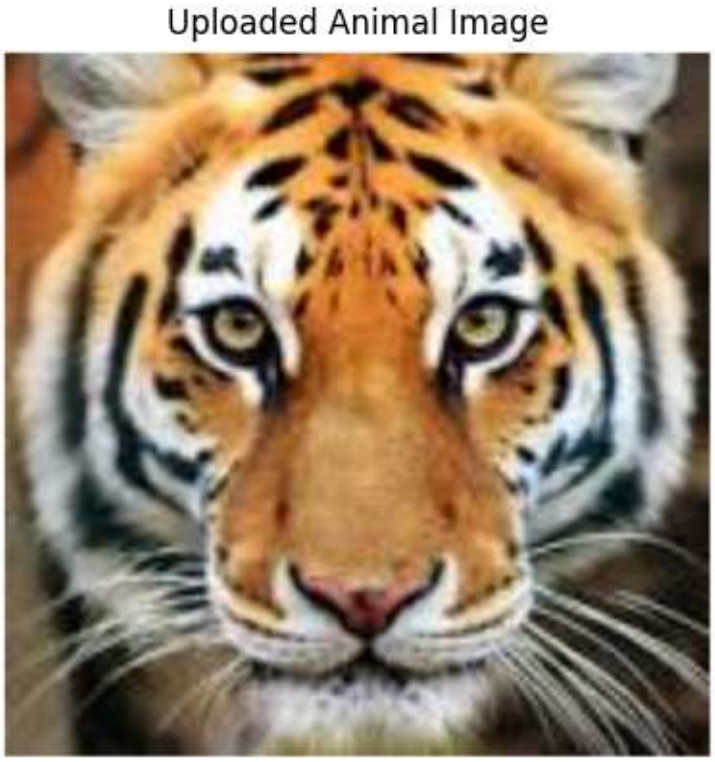
Top 3 Predictions:

1. Egyptian cat: 53.35%

2. tabby: 25.17%

3. tiger cat: 8.29%


Uploaded image: Screenshot 2026-07-25 092012 (2).png




**1/1** ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **2s** 2s/step

Top 3 Predictions:

1. tiger: 72.15%

2. tiger cat: 17.45%

3. jaguar: 0.67%